



CSC432 – INFORMATION SECURITY

Dr. Muhammad Sharjeel

muhammadsharjeel@cuilahore.edu.pk



Lecture No 10

KEY DISTRIBUTION AND MANAGEMENT

Key Distribution and Management



At many points so far, we have run into problems with using cryptography for computer security

- ☐ How can we securely store and distribute keys (and password)?
- ☐ How can we certify that keys are safe and to be trusted?
- ☐ To solve these issues, we make use of a variety of trusted intermediaries

Key Distribution and Management



Pros and Cons

- ❑ Symmetric key cryptography: fast implementations, good for encrypting large amounts of data; requires shared secret key
- ❑ Asymmetric (public) key cryptography: inefficient for large data, good for authentication; no need to share a secret

Issues:

How to share symmetric keys?

How to distribute public keys?

Key Distribution and Management



Symmetric key problem:

- ❑ How do two entities establish shared secret key over a computer network?

Solution: Trusted Key Distribution Centre (KDC) acting as intermediary between entities

Asymmetric key problem:

- ❑ When Alice obtains Bob's public key (from web site, e-mail, etc.), how does she know it is Bob's public key, not Trudy's?

Solution: Trusted Certification Authority (CA)

Key Distribution and Management



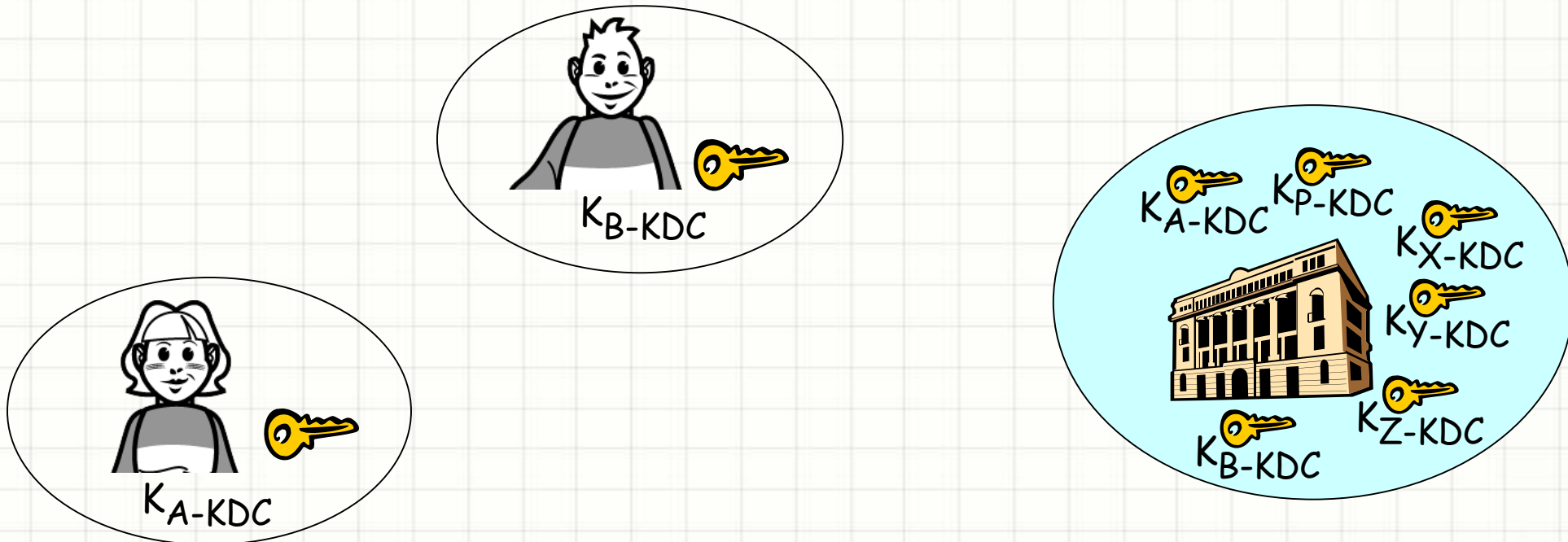
Key Distribution Centre

- ☐ If A and B have secure connection with third party C, C can securely send keys to A and B
- ☐ Key Distribution Centre (KDC) is trusted third party

Key Distribution Centres (KDCs)



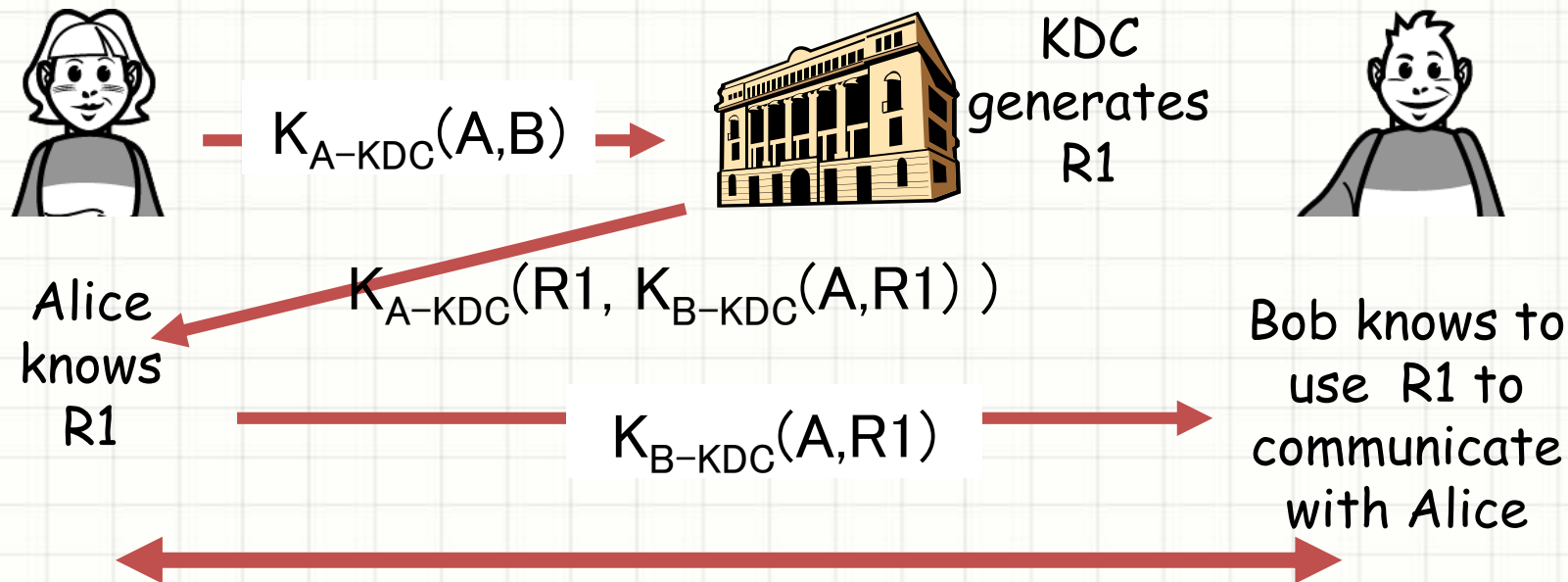
- ❑ Alice, Bob need shared symmetric key
- ❑ KDC server shares different secret keys with each registered user (many users)
- ❑ Both Alice and Bob know their own symmetric keys (master keys), K_{A-KDC} and K_{B-KDC} , for communicating with KDC



Key Distribution Centres (KDCs)



- How does KDC allow Bob, Alice to determine shared symmetric secret key to communicate with each other?



Alice and Bob communicate: using $R1$ as session key for shared symmetric encryption

Key Distribution Centres (KDCs)



- ❑ How are the keys shared with the KDC originally agreed upon and distributed?
 - ❑ KDC has all the keys, while each node is configured with KDC's key (master key)
 - ❑ Most approaches assume some form of manual key generation and distribution with the KDC
- ❑ The KDC is a critical, centralized part of network infrastructure
 - ❑ If KDC is compromised, all systems are compromised
 - ❑ KDC is single point of failure or performance bottleneck
 - ❑ This makes the KDC subject to interruption attacks to disrupt the general security of the network
- ❑ How does this approach scale?
 - ❑ A single KDC cannot service all users of the Internet!
 - ❑ To scale, we require a collection of KDCs, that work together for key distribution

Key Distribution Centres (KDCs)



A Typical Network Scenario

- ☐ When a user Alice first logs into a workstation, she'll type her account name and password
- ☐ We'll call the period from when she logs in to when she logs out her login session
- ☐ During her login session Alice will probably need to access remote resources
- ☐ These remote resources will need to authenticate her, but Alice's workstation performs the authentication (using protocol) on Alice's behalf, and Alice need not be aware that it is happening

Kerberos: A KDC in Practice



- ❑ Kerberos is an authentication system developed in 1988 at MIT as part of Project Athena
- ❑ Based on Needham and Schroeder's secret key protocol, with DES to handle encryption and decryption
- ❑ Kerberos is used to provide secure access, protects against eavesdropping and replay attacks
- ❑ Used in Windows as default authentication mechanism, Apple's Mac OS X clients and servers also use Kerberos
- ❑ Apache HTTP Server, Eudora, NFS, OpenSSH, rcp , rsh allow using Kerberos for authentication
- ❑ There are two commonly available versions of Kerberos, version 4 and version 5 (RFC 4120)

Kerberos: A KDC in Practice



- ❑ At the heart of Kerberos is its own version of a Key Distribution Centre
- ❑ KDC runs on a physically secure node somewhere on the network, there is a library of subroutines used by distributed applications which want to authenticate their users
- ❑ When a user logs into a workstation by providing a user name and a password, the workstation used them to obtain information from the KDC that can be used by any process to access remote resources on behalf of the user

Kerberos: A KDC in Practice



- ❑ In Kerberos a KDC is logically comprised of two services:
- ❑ Authentication Service (AS): authenticates user clients and provides access to the TGS
- ❑ Ticket Granting Service (TGS): provides tickets and session keys to allow users to access particular services within the Kerberos realm

Kerberos: A KDC in Practice



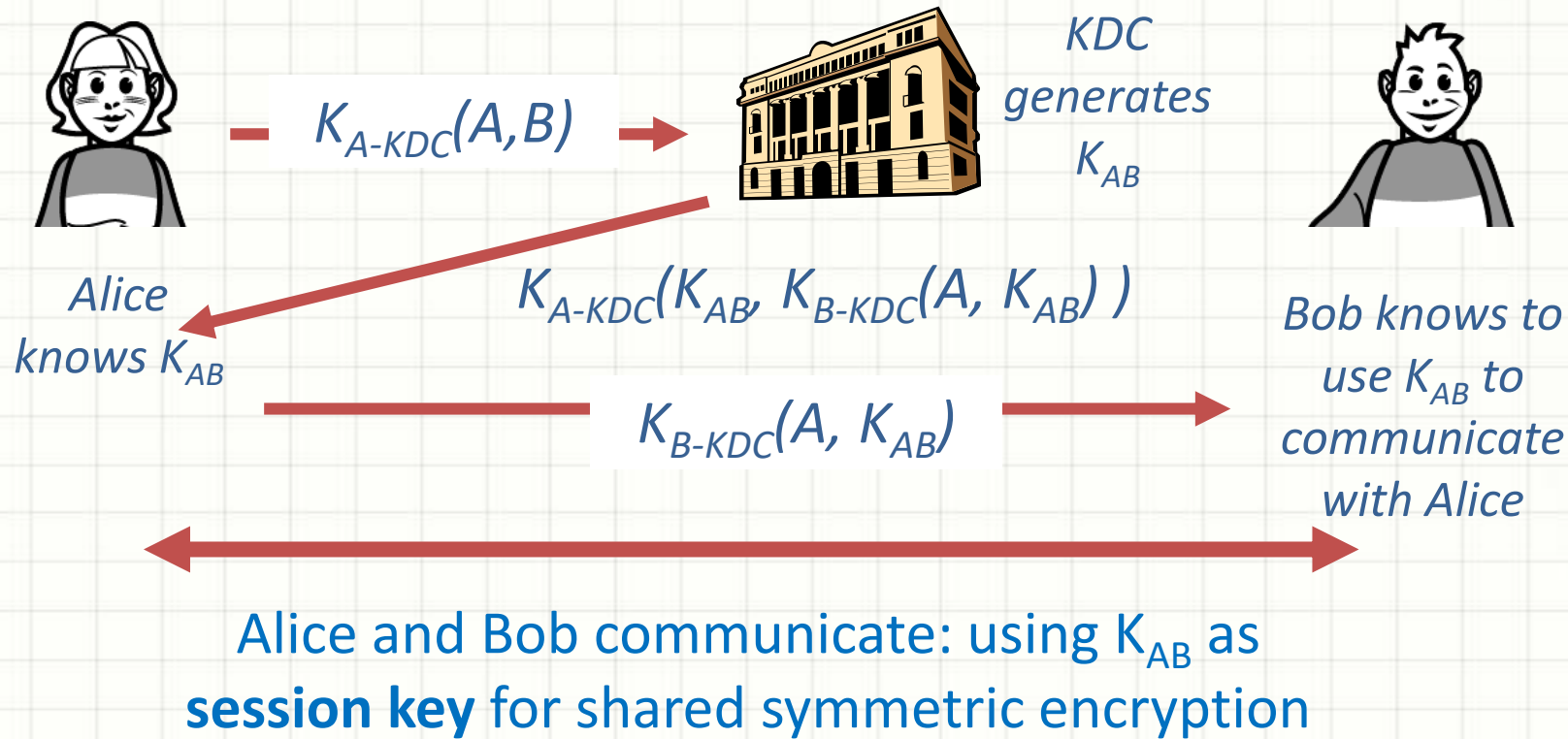
- ❑ KDC in Kerberos shares a secret key, known as a master key, with each user and each resource that will be using Kerberos
- ❑ When Alice informs the KDC that she wants to talk to Bob, the KDC invents a session key K_{AB} for Alice and Bob to share, encrypts K_{AB} with Alice's master key for Alice, encrypts K_{AB} with Bob's master key for Bob, and returns all this information to Alice
- ❑ The message consisting of the session key K_{AB} (and other information such as Alice's name) encrypted with Bob's master key is known as a ticket to Bob
- ❑ Alice can't read what's inside the ticket, because it's encrypted with Bob's master key

Kerberos: A KDC in Practice



- ❑ Bob can decrypt the ticket and discover K_{AB} and Alice's name
- ❑ Bob knows, based on the ticket, that anyone else who knows K_{AB} is acting on Alice's behalf
- ❑ So Alice and Bob can authenticate each other, and optionally encrypt or integrity-protect their entire conversation, based on the shared key K_{AB}
- ❑ The session key K_{AB} together with the ticket to Bob are known as Alice's credentials to Bob

Kerberos: A KDC in Practice



Kerberos: A KDC in Practice



- ❑ All communication in Kerberos is protected through symmetric key cryptography (DES)
- ❑ Each client (user) and server in Kerberos has a secret key known only by the individual and the Kerberos Authentication Service
 - ❑ To facilitate the process, users have passwords that are transformed to their secret key when needed
- ❑ For communication between everything else, session keys are used that generated by the Ticket Granting Service
- ❑ Tickets are tokens issued to clients to present to a particular server to verify that the client was recently authenticated by Kerberos
- ❑ Each ticket contains an expiry time and a session key for use by the client and the particular server

Kerberos: A KDC in Practice



- ❑ Kerberos protects against eavesdropping:
 - ❑ If someone else sends TGS, they get back a ticket, and can't decrypt the service key unless they know the client's secret key
- ❑ Kerberos protects against replay attacks:
 - ❑ If someone sends TGS ticket after a while, it is rejected
 - ❑ All clients, servers should have time synchronized within a specified limit
- ❑ Ticket's contain requesters IP address, no one else can use the ticket without changing their IP addresses

Kerberos: A KDC in Practice



- ❑ Through its ticket-granting capabilities, one can achieve scalability by having one KDC grant a ticket permitting a client to contact a KDC in another realm
- ❑ Most KDC operations are read-only
- ❑ But, we still have issues of setting up secret keys in the first place, and the potential for interruption attacks on the KDC itself
- ❑ Furthermore, Kerberos depends on a good network time service to synchronize system times for accepting timestamps
- ❑ This makes the time service a good target for interruption attacks too!

Distribution of Public Keys



- ☐ By design, public keys are made public

Issue: how to ensure public key of A actually belongs to A (and not someone pretending to be A)

Approaches for distributing public keys

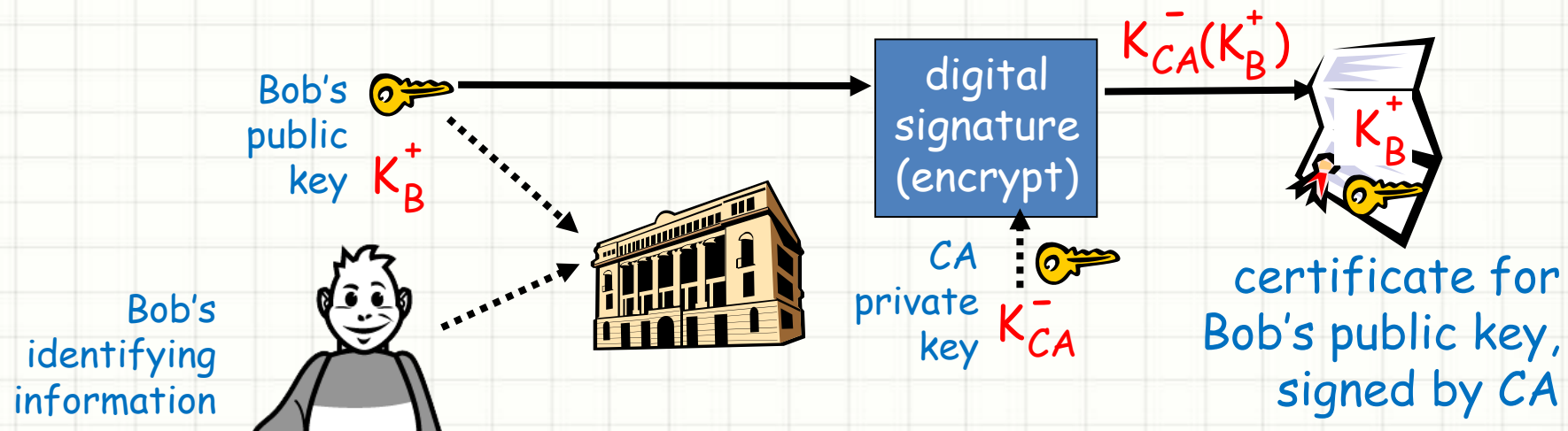
- ☐ Public announcement
- ☐ Publicly available directory
- ☐ Public-key certificates

Certification Authorities



- ❑ Unsigned public keys can be tampered
- ❑ Certification Authority (CA) binds a public key to particular entity, E
 - ❑ E (person, router) registers its public key with CA
 - ❑ E provides “proof of identity” to CA
 - ❑ CA creates certificate binding E to its public key
 - ❑ Certificate containing E’s public key digitally signed by CA, CA says “this is E’s public key”

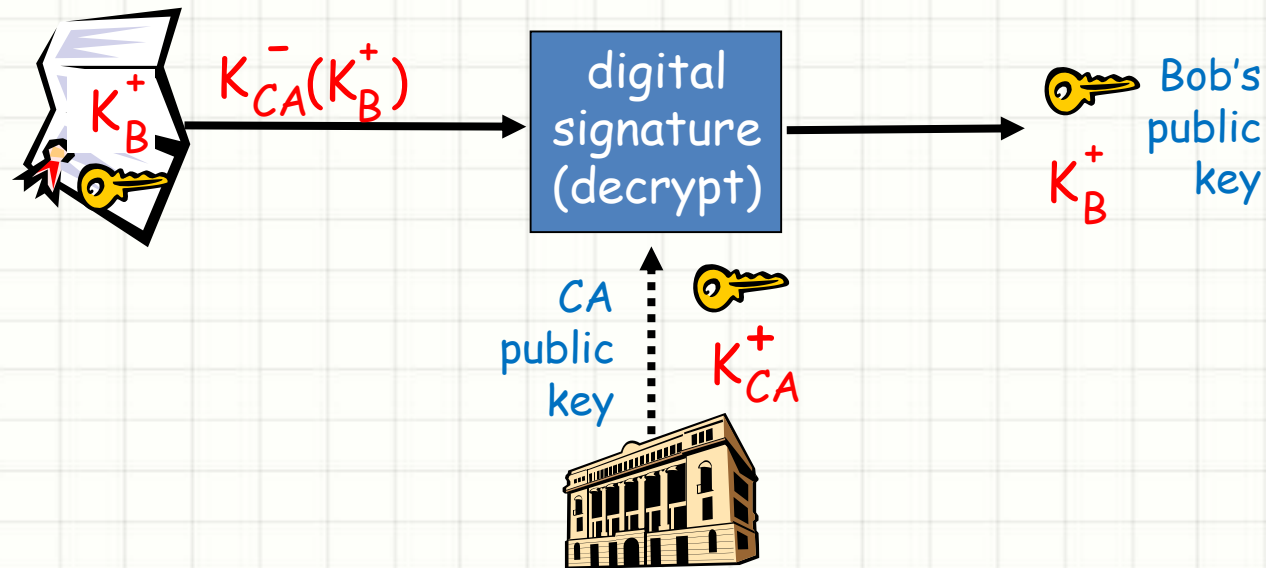
Certification Authorities



Certification Authorities



- When Alice wants Bob's public key:
 - Gets Bob's certificate (Bob or elsewhere)
 - Apply CA's public key to Bob's certificate, get Bob's public key



Certification Authorities



But, how does Alice get the authority's public key?

- ☐ Assume they are published in a trusted, public, and well-known place. But, how can you really trust this?
- ☐ Pre-configure trusted Certificate Authorities into the software requiring them
- ☐ For example, web browsers come pre-configured to recognize many authorities, and have the facilities to add or accept new ones as necessary

How can you really trust a Certificate Authority?

- ☐ In many domains, these authorities are run, licensed, or regulated by governments or government-appointed bodies
- ☐ Ultimately it is up to the individual receiving a certificate to judge whether they accept it or not
- ☐ Keep in mind, though, that refusing to accept an authority means that you cannot communicate securely with other individuals that use it!

References



- ❑ Wikipedia articles

Key Distribution Centre, http://en.wikipedia.org/wiki/Key_distribution_center

Certification Authority, http://en.wikipedia.org/wiki/Certificate_authority

- ❑ Kerberos: The Network Authentication Protocol

<http://web.mit.edu/kerberos/>

- ❑ “Cryptography and Network Security: Principles and Practice”, 6th edition, by William Stallings (chapter 14)

- ❑ “Network Security: Private Communication in a Public World”, by Charlie Kaufman, Radia Perlman, Mike Speciner (chapter 9)



THANKS